第 20 章練習解答　具影響力的作業系統

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS20-E01　spool真瓶頸

1. CPU J1 0–2/J2 2–4/J3 4–6；printer J1 2–6/J2 6–10/J3 10–14。後續輸出可先排隊，不讓J2打斷J1printer。
2. 總14，CPU6/14=3/7，printer12/14=6/7；完全串行3×(2+4)=18。單printer工作仍12，不會因有3jobs縮成4。
3. spool保存jobs資料／queue使生產消費解耦；multiprogramming另讓等待者之外的ready工作跑CPU。前者不自動保證RAM已有多jobs、隔離與排程。

檢查。 給完整隊列與時間，不只說提高利用率。

OS20-E02　THE與Mach的邊界

1. 都分解服務與責任，但THE重在逐層抽象／同步，Mach3 user server另以受保護address space與IPC提供邊界。軟體模組化不自動等於故障隔離。
2. 只有已指定服務位於另一user server、走IPC時才能據此計AS切換。THE layer呼叫不必trap；Mach2.x大量BSD仍在kernel，不能套Mach3全部規則。
3. C0成立；Ci在下層C0…C(i−1)與明確干擾前提下建立且不破壞下層；以歸納推各層。cycle或直接改下層隱藏狀態使前提失效。

檢查。 不把layer、ring、process混為一談。

OS20-E03　Atlas三分支與現代fault

1. 第一組全t0，用③Tmax選B。第二組無t>T+1，②只看nonzero t，A score2、B5，選B，忽略C0。若漏③，第一組合法狀態沒有定義victim。
2. 3597=7×512+13，physical word address=2×512+13=1037；不把word當byte。
3. 共同需有效mapping、內容正確、fault後可重啟與frameownership；現代多層表、TLB、COW、共享／權限及多core同步增加條件。Atlas過去間隔預測不保證OPT最低fault。

檢查。 列全t0邊界，不只概述歷史。

第 20 章練習解答　具影響力的作業系統 （續）

OS20-E04　XDS-940 relocation及共享

1. 16/2=8virtual pages，64/2=32frames；6161=3×2048+17→11×2048+17=22545word address。
2. 2shared+2×6private=14frames，而非16或12。需code唯讀／reentrant且可變資料分開；多個映射不會自動複製共享資料。
3. 否。paging可只做relocation，這正是此歷史案例；缺頁按需載入、fault重啟與swapping policy是額外機制。

檢查。 單位與駐留政策分開。

OS20-E05　RC4000式buffer守恆

1. 兩req送完free空、queued{b0,b1}；S處理並回b0後free仍空、queued{b1}、reply{b0}，總2。processing狀態若尚未reply則把b0單獨放processing，不可重複列。
2. 不能，b0仍由reply持有。C1取reply且協定完成歸還後free{b0}，C3才可claim；buffer恰屬一個ownership狀態。
3. 不能。若C1等S答、S等C1另訊息會循環等待。atomic send只使該狀態轉移不可分割，不保證全request/reply交易與全系統進度。

檢查。 守恆與等待關係分別檢查。

OS20-E06　指數feedback真軌跡

1. 允許1/2/4/8，實際1/2/4/4，剩餘10/8/4/0；4次dispatch。最後不是一定用滿8才結束。
2. feedback CPU11+4×0.5=13；RR11次dispatch為11+5.5=16.5。固定成本模型，不是歷史實測加速。
3. 從0至k的完整量1+2+⋯+2^k=2^(k+1)−1。若中間阻塞／提升／多工作插入，單一burst軌跡與waiting需重算；指數量子也不自行防低priority飢餓。

檢查。 固定規則後才計算，勿混不同MLFQ。

OS20-E07　Multics segment與page

1. 1024=2^10，offset的18bits拆8bit page index＋10bit offset；2500=2×1024+452，PA=7×1024+452=7620word。segment descriptor的bound/rights仍須先過。
2. ring是受硬體／入口規則強制的privilege邊界；layer是軟體依賴／抽象結構。module可同ring，單畫六層不證明六個保護domain。
3. 解析只取得物件reference／身分，還需access list、requested operation及執行環境允許。共享可寫資料仍需同步，不能因segment有名稱就忽略權限。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 20 章練習解答　具影響力的作業系統 （續）

檢查。 每層mapping與權限檢查分開。

OS20-E08　MFT/MVT與base-limit

1. 固定兩50各占64，internal waste28KiB；剩兩region共128仍無單一region容70。variable總holes120卻最大50不能容70，external fragmentation。
2. 先驗0≤a<768，0→PA256、767→1023，768/−1拒。base/limit必由受保護機制設置；若user可改就失去隔離。
3. CP提供VM資源隔離，CMS在其上以single-user系統方式運行；多guest不是CMS自己成為共享全部memory的multiuserkernel。配置版本、relocation與VM不同。

檢查。 不拿總free量取代連續區間。

OS20-E09　Mach3指定copy模型

1. request Cuser→K→Suser為2mode/1AS/64B；reply同理2mode/1AS/8192B；總4mode、2AS、8256copyB。未計其他IRQ或scheduler活動，若模型改了須另算。
2. memory protection可限制server直接覆寫client／kernel；仍需client處理timeout、失效狀態、重新連線／重試。不能推得服務永不故障或kernel沒有bug。
3. Mach2.x大量BSDcode仍在kernel；後續衍生系統可重新組合kernel/user功能。版本與實際服務位置未定，不能只因含Mach名字就算全部IPC成本。

檢查。 分開模式、AS、資料byte計數。

OS20-E10　歷史概念不等於表面名稱

1. cooperative若只在yield/block切換，B可永遠不跑；preemptive若timer可服務、kernel阻塞有界且公平選取，B才有進度。兩視窗不證明多process或memory隔離。
2. 不能。語法／介面輔助只建立可辨識request，核心仍需以可信身分、目標物件、requested right判定；名字也可能在check/open間替換。
3. 不能。語言可提高表達／檢查能力，小kernel可縮某些受信任範圍，仍須維持同步、ownership、權限與failure契約；每種結構保證其已證明的部分。

檢查。 把可用性、隔離、進度及一般正確性拆開。

第 20 章練習解答　具影響力的作業系統 （續）

原創驗算程式附錄: history_models.py

```
"""Self-contained historical mechanism models; not historical machine emulation."""
from collections import deque


def atlas_victim(pages):
    """Each row=(name,t,T), nonnegative sampled history. First row breaks ties."""
    if not pages or any(t<0 or T<0 for _,t,T in pages):raise ValueError('nonempty valid
history')
    if len({p[0] for p in pages})!=len(pages):raise ValueError('duplicate page')
    overdue=[p for p in pages if p[1]>p[2]+1]
    if overdue:return overdue[0][0],1
    inactive=[p for p in pages if p[1]!=0]
    if inactive:return max(inactive,key=lambda p:p[2]-p[1])[0],2
    return max(pages,key=lambda p:p[2])[0],3


def feedback(burst,initial=0):
    """q_i=2**i; no competitor, I/O, promotion or cost in timeline."""
    if burst<0 or initial<0:raise ValueError('negative work/level')
    remaining=burst;level=initial;total=0;rows=[]
    while remaining:
        q=2**level;used=min(q,remaining);remaining-=used;total+=used
        rows.append((level,q,used,total,remaining));level+=1
    return rows


def spool(jobs,cpu=2,printer=4):
    if jobs<0 or cpu<=0 or printer<=0:raise ValueError('invalid workload')
    cpuend=0;printend=0;rows=[]
    for job in range(1,jobs+1):
        start=cpuend;cpuend+=cpu;printstart=max(cpuend,printend);printend=printstart+printer
        rows.append((job,start,cpuend,printstart,printend))
    return rows


class Pool:
    """One buffer per request+reply exchange; deterministic atomic transitions."""
    def __init__(self,count=2):
        if count<0:raise ValueError('count')
        self.state={i:'free' for i in range(count)};self.queue=deque()
    def send(self):
        free=[i for i,s in self.state.items() if s=='free']
        if not free:raise BlockingIOError('pool exhausted')
        i=min(free);self.state[i]='queued';self.queue.append(i);return i
    def receive(self):
        if not self.queue:raise BlockingIOError('no request')
```

第 20 章練習解答　具影響力的作業系統 （續）

```
        i=self.queue.popleft();self.state[i]='processing';return i
    def answer(self,i):
        if self.state.get(i)!='processing':raise ValueError('not server-owned')
        self.state[i]='reply'
    def reclaim(self,i):
        if self.state.get(i)!='reply':raise ValueError('no completed reply')
        self.state[i]='free'
    def valid(self):
        return len(set(self.queue))==len(self.queue) and set(self.queue)=={i for i,s in
self.state.items() if s=='queued'} and all(s in ['free','queued','processing','reply'] for s
in self.state.values())


def split_ge645(segment, word_offset):
    """Original GE645 two18-bit fields; segment-bound/protection checks separate."""
    if not 0<=segment<(1<<18) or not 0<=word_offset<(1<<18):
        raise ValueError('GE645 field outside18bits')
    return segment,word_offset//1024,word_offset%1024


def ge645_read(segment,word_offset,descriptors):
    """Paged-segment example, authority checks before demand-page resolution.
    Not a complete descriptor encoding/GE645 machine implementation.
    """
    number,page,within=split_ge645(segment,word_offset)
    desc=descriptors.get(number)
    if desc is None or page>=desc['bound_pages'] or not desc['read']:
        return 'protection-fault',None
    pte=desc['pages'].get(page)
    if pte is None or not pte['legal'] or not pte['read']:
        return 'protection-fault',None
    if not pte['present']:return 'page-fault',None
    return 'success',pte['frame']*1024+within


def protected_address(a,base=256,limit=768):
    if not 0<=a<limit:raise PermissionError('outside user segment')
    return base+a


if __name__=='__main__':
    for case in [[('A',6,3),('B',2,5),('C',0,9)],[('A',2,4),('B',1,6),('C',0,8)],[('A',0,2),
('B',0,5),('C',0,3)]]:print('Atlas',atlas_victim(case))
    print('feedback11',feedback(11));print('spool3',spool(3))
    p=Pool();a=p.send();b=p.send();i=p.receive();p.answer(i)
    print('before reply consume',p.state);p.reclaim(i);print('reclaimed',p.state,p.valid())
```

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study